
ProjectName – Java

Rapport final

Étudiant1, Étudiant2, Étudiant3, Étudiant4, Étudiant5

7 avril 2026

1 L'Algorithme Minimax :

Conçu pour des jeux à information parfaite et à somme nulle comme les Échecs, l'algorithme Minimax modélise l'arbre des coups possibles en alternant un tour où l'IA cherche à maximiser son score, et un tour où elle simule un adversaire cherchant à minimiser l'avantage de l'IA.

Pour adapter cet algorithme au Scrabble (où le chevalet adverse est inconnu), notre implémentation déduit d'abord les tuiles restantes en soustrayant du jeu complet les lettres déjà posées sur le plateau et celles présentes dans le chevalet de l'IA. Ensuite, le solveur utilise une méthode de Monte-Carlo : il simule l'adversaire en tirant aléatoirement 200 chevalets virtuels de 7 lettres parmi ces tuiles restantes. Le Minimax évalue alors le pire scénario possible : parmi ces 200 simulations, il retient le coup adverse qui marque le plus de points, et le soustrait du score du coup envisagé par l'IA.

L'impact de l'heuristique

Au Scrabble, évaluer un coup uniquement par les points qu'il rapporte sur le plateau n'est pas le plus pertinent. L'algorithme intègre donc une fonction heuristique d'évaluation qui note la qualité des lettres conservées sur le chevalet après un coup. Cette heuristique donne des bonus et malus en fonction de :

- Un joker conservé rapporte un bonus massif de +15.0 points.
- La lettre 'S' rapporte +8.0 points.
- Les lettres difficiles (J, K, Q, X, Z) appliquent un malus de -6.0 points.
- L'équilibre voyelles/consonnes est récompensé (+5.0 points) tandis qu'un chevalet exclusivement composé de voyelles ou de consonnes est lourdement pénalisé (-12.0 points).

Complexité et coût

Dans notre implémentation de Monte-Carlo, l'algorithme ne disposant pas d'élagage Alpha-Beta, la complexité temporelle est strictement identique à celle de l'Expectiminimax, soit $O(B \times N \times B)$ pour une anticipation d'un tour de l'adversaire (où B est le facteur de branchement et $N = 200$ le nombre de tirages simulés). La complexité reste modérée car le parcours se fait en profondeur d'abord. Le coût est massif puisqu'il doit évaluer exactement les 200 scénarios complets. L'exécution est donc bridée par une limite de temps stricte (`timeLimitMs`), l'empêchant de calculer sur plusieurs tours d'anticipation.

Performances

La performance défensive est excellente. En s'attendant systématiquement au pire, l'IA verrouille le plateau et évite d'ouvrir des cases multiplicatrices dangereuses. Mais ce pessimisme absolu la rend parfois trop peu entreprenante sur le plan offensif. De plus, puisqu'elle paie le même coût que l'Expectiminimax sans bénéficier d'une prise de décision nuancée, cette implémentation se révèle, à temps de calcul égal, moins optimale que l'approche probabiliste.

2 L'Algorithme Expectiminimax :

L'Expectiminimax est l'évolution du Minimax pour les jeux intégrant du hasard. Il introduit un nouveau type de nœud dans l'arbre de recherche : les "nœuds de chance" (chance nodes). Au lieu de considérer que l'adversaire jouera le pire coup absolu, l'algorithme calcule l'espérance mathématique de tous les coups possibles en fonction de leur probabilité.

Dans notre code, la méthode `expectiminimax()` utilise la même base de simulation que le Minimax (les 200 tirages aléatoires de chevauxets adverses). Cependant, au lieu d'extraire la valeur maximale de ces simulations, l'algorithme calcule la moyenne des meilleurs scores adverses obtenus sur ces 200 échantillons. Il évalue ainsi un coup en soustrayant cette "réponse moyenne attendue" du score du coup de l'IA.

L'impact de l'heuristique

L'Expectiminimax ne se contente pas de calculer mathématiquement les risques sur le plateau. Pour juger de la force réelle d'un coup, l'IA additionne deux éléments concrets :

- **Le risque sur le plateau (L'algorithme)** : L'IA vérifie qu'elle ne laisse pas d'opportunités trop faciles et dangereuses à l'adversaire (comme ouvrir l'accès à une case "Mot Compte Triple").
- **La qualité de sa main (L'heuristique)** : L'IA évalue les lettres qu'elle garde sur son chevalot pour son prochain tour.

C'est le compromis entre ces deux visions qui rend l'IA véritablement intelligente. Par exemple, si un coup sécurise parfaitement le plateau mais oblige l'IA à conserver une main catastrophique, l'heuristique va agir comme un garde-fou et lourdement pénaliser ce choix. L'IA cherche donc toujours l'équilibre parfait entre "limiter la casse sur le plateau" et "garder de bonnes lettres pour l'avenir".

Complexité et coût

La complexité est de $O(B \times N \times B)$, ce qui provoque une explosion combinatoire. Bien que son coût soit strictement identique à celui de notre Minimax actuel, l'Expectiminimax a l'inconvénient théorique de ne presque pas pouvoir être optimisé par des techniques d'élagage (comme l'Alpha-Beta). En effet, le calcul d'une espérance mathématique exige d'explorer toutes les probabilités jusqu'au bout, ce qui le coupe quasi systématiquement avant d'atteindre une profondeur de recherche de 2.

Performances

Sur le papier et dans notre code actuel, c'est la meilleure performance stratégique pour le Scrabble. L'évaluation du risque est mieux gérée : l'IA n'hésitera pas à ouvrir un "Mot Compte Triple" si elle calcule que la probabilité que l'adversaire possède les lettres exactes pour s'y placer est très faible. Le style de jeu devient plus naturel. Étant donné que son coût est identique à celui du Minimax dans notre code, l'Expectiminimax s'avère supérieur dans la prise de ses décisions.

3 Comparaison des deux Algorithmes

Les schémas ci-dessous mettent en évidence que la différence entre nos deux algorithmes n'est pas structurelle (l'arbre exploré est identique), mais réside uniquement dans la méthode d'agrégation des nœuds de simulation.

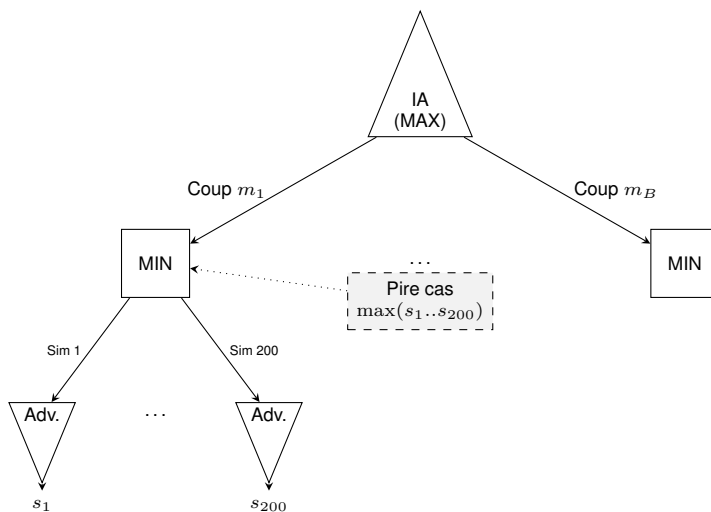


FIGURE 1 – Agrégation Pessimiste

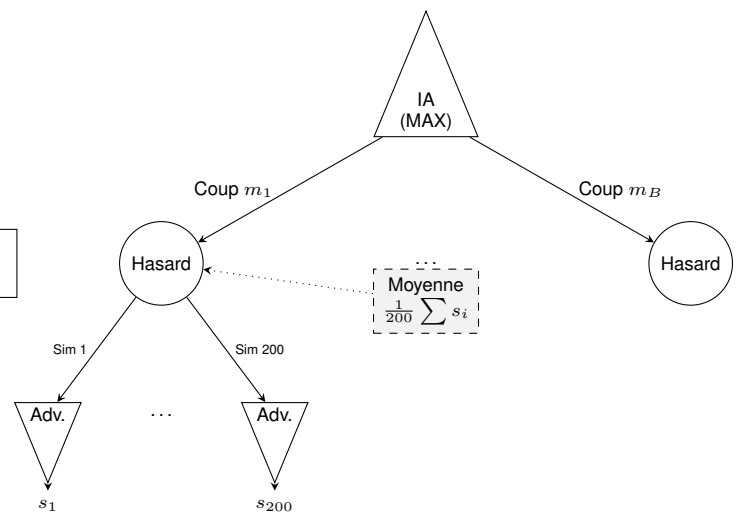


FIGURE 2 – Agrégation Probabiliste

Analyse des performances

Afin de confronter notre étude théorique à la réalité matérielle, nous avons instrumenté le code de notre solveur (`MinimaxSolver.java`) pour extraire des métriques physiques en conditions réelles de jeu. Le chronomètre d'interruption (`timeLimitMs`) était fixé à 5000 millisecondes.

Le tableau ci-dessous synthétise des échantillons représentatifs de la prise de décision de l'IA au cours d'une partie :

Algorithme	Temps	Statut	Coups IA Évalués	Simulations Adv.	Moy. Sim/Coup
Expectiminimax	5000 ms	Timeout	48 sur 84 (57%)	9 497	197
Expectiminimax	3575 ms	Complet	33 sur 33 (100%)	6 600	200
Minimax MC	5001 ms	Timeout	24 sur 58 (41%)	4 680	195
Minimax MC	5001 ms	Timeout	17 sur 41 (41%)	3 294	193
Minimax MC	2801 ms	Complet	10 sur 10 (100%)	2 000	200

TABLE 1 – Mesures physiques des algorithmes (Profondeur 2, SAMPLES_COUNT = 200)

Interprétation des résultats

L'analyse de ces logs d'exécution met en évidence trois points cruciaux concernant les limites de notre implémentation :

- **La limite du facteur de branchement** : Les données prouvent que la capacité de l'IA à évaluer l'arbre complet dépend entièrement de la complexité du plateau à l'instant T. Lorsque le nombre de mots possibles est faible (ex : 10 ou 33 coups), le moteur termine son parcours de Profondeur 2 confortablement avant le temps limite (2.8s et 3.5s). À l'inverse, dès que les choix s'élargissent (41, 58 ou 84 coups), l'IA est systématiquement bridée.
- **Les conséquences du coût élevé** : Dans le pire scénario mesuré (17 coups évalués sur 41), le *Timeout* force l'IA à jouer en ignorant totalement 59% des opportunités légales présentes sur le plateau, l'empêchant potentiellement de trouver le mot optimal.
- **L'efficacité de la limite de temps** : Le moteur est capable de simuler un volume massif de plateaux adverses (jusqu'à 9 497 en 5 secondes). Lorsque le *Timeout* est déclenché, on observe que la moyenne des simulations tombe légèrement sous la barre des 200 (192, 195, 197). Cela démontre que notre algorithme d'interruption fonctionne avec précision : il avorte la boucle interne de Monte-Carlo au milieu de l'évaluation du N -ième coup pour garantir la fluidité du jeu en temps réel.

Ces mesures empiriques confirment de manière irréfutable notre conclusion : bien que la Profondeur 2 soit mathématiquement atteinte, l'absence d'élagage dans nos approches de Monte-Carlo bride physiquement la rentabilité de l'arbre dès que le facteur de branchement dépasse la trentaine de mots candidats.

4 Le Machine Learning :

Contrairement aux algorithmes de recherche arborescente, le Machine Learning propose une approche statistique. Notre implémentation utilise un réseau de neurones conçu avec TensorFlow. L'architecture du modèle traite la recherche de mots comme un problème de classification multiclasse :

- **L'entrée** : Le chevalet du joueur est converti en un tenseur de 26 dimensions, représentant la fréquence de chaque lettre (de A à Z).
- **Les couches cachées** : Deux couches denses de 256 et 512 neurones analysent les motifs et anagrammes.
- **La sortie** : Une couche Softmax fournit une distribution de probabilités sur l'ensemble du dictionnaire (une probabilité par mot existant).

En jeu, l'agent Java interroge ce modèle pré-entraîné, filtre les mots ayant une probabilité de faisabilité supérieure à 0,1%, et trie les 100 meilleures prédictions.

Complexité et coût

La complexité est divisée en deux parties. La phase d'entraînement a une complexité linéaire dépendante de la taille du dictionnaire, du nombre d'époques et de la taille des lots. En partie, la complexité d'inférence est constante, soit $O(1)$. Le coût en jeu est faible (quelques millisecondes pour une matrice tensorielle). En revanche, le coût en mémoire vive est élevé en raison du chargement natif des bibliothèques TensorFlow et du graphe du modèle dans l'environnement Java.

Performances

La performance pure est très efficace pour trouver des anagrammes complexes. Mais la performance de jeu est nulle de manière autonome : le réseau actuel ignore totalement l'état du plateau et les contraintes de placement. Le moteur doit d'ailleurs prévoir une redirection vers les algorithmes arborescents si aucun des mots prédits par l'IA ne peut être légalement posé.

Conclusion :

La comparaison de ces méthodes met en évidence qu'aucun algorithme n'est parfait s'il est utilisé de manière isolée. L'Expectiminimax est le champion de la stratégie probabiliste, mais son coût le rend peu intéressant pour des recherches profondes. Le Minimax classique est efficace, surtout avec une bonne heuristique, mais excessivement défensif face à l'inconnu, et notre implémentation le prive de son avantage temporel théorique. Le Machine Learning est un très bon calculateur d'anagrammes, mais il est "aveugle" aux règles de placement sur le plateau.

La solution la plus efficace réside dans la fusion de ces approches. La complexité de l'Expectiminimax provient de son facteur de branchement immense (générer tous les mots possibles). En utilisant le Machine Learning comme un outil d'élagage probabiliste, le modèle TensorFlow peut instantanément suggérer les 20 meilleurs mots possibles depuis le chevalet. L'Expectiminimax n'a plus qu'à évaluer ces 20 branches réduites face aux risques du plateau, obtenant ainsi la certitude mathématique et probabiliste tout en respectant un temps de calcul extrêmement restreint.

Première partie

Spécifications Étendues

5 F21 — Sauvegarde et restauration d'une partie

Description

Le programme doit pouvoir sérialiser une partie en cours dans un fichier texte ASCII et reconstruire une partie complète à partir d'un fichier de sauvegarde. Le parseur doit être robuste aux erreurs de format et les signaler avec précision (type d'erreur et numéro de ligne). L'implémentation repose sur les classes `SaveManager.java` et `GameLoader.java`.

Prérequis

- **F2 — Configuration** : Paramètres par défaut (langue, blitz, etc.) via `.scrabblerc` — `ConfigLoader.java`
- **F10 — Règles du jeu** : État complet du jeu : plateau, scores, joueurs, historique, tour courant
- **F15 — Shell interactif** : Commandes `save FILE` et `load FILE` — `GameControllerAux.java`
- **F22 — Commentaires** : Gestion des commentaires `#` et blocs `{ }` — `GameLoader.java`
- **F23/F24/F25 — Formats** : Sections `[settings]`, `[game]` et `[history]` du fichier de sauvegarde

Type de besoin

Fonctionnel.

Sous besoins 1. Sauvegarder la partie dans un fichier

Description : S rialiser l tat courant du jeu dans un fichier texte ASCII en  crivant successivement les trois sections obligatoires.

Fonction associ e :

```
void saveGame(Game game, String filePath) throws IOException
```

Actions :

- Ouvrir (ou cr er) le fichier au chemin indiqu .
-  crire successivement [settings], [game] puis [history].
- Fermer le fichier proprement.
- Remonter IOException en cas d chec (g r e par CLI ou GUI).

R sultat attendu :

- *Succ s* : fichier complet et bien forme  crit sur le disque.
- *Ech c* : exception IOException remont e, aucun fichier partiel conserve.

Test unitaire :

- **Cas 1 :** Entr e :  tat complexe (plateau avec mots, 2 joueurs, scores, racks, coups pass/play). Attendu : fichier cr e avec 3 sections, v rifi e via SaveManagerTest.java.
- **Cas 2 :** Entr e : conversion de coordonn es a1v / h8h. Attendu : format des coordonn es correct dans [history].
- **Cas 3 :** Entr e : partie vide (aucun coup jou ). Attendu : plateau serialis e en lignes de tirets, section [history] vide.

Sous besoins 2. Serialiser la section [settings]

Description :  crire les param tres globaux de la partie et les param tres IA joueur par joueur sous la section [settings].

Param tres  crits :

- blitz — mode blitz active ou non.
- super-scrabble — plateau 21 21.
- players-count — nombre de joueurs.
- debug / verbose — niveaux de trace.
- player-N-type — human ou ai.
- player-N-ai-mode — algorithme IA (ex. : minimax).
- player-N-name — nom du joueur.

R sultat attendu :

- Section [settings] complete et coh rente avec la configuration de partie.

Sous besoins 3. Serialiser la section [game]

Description :  crire l tat du plateau, les racks et les scores de chaque joueur.

Fonctions internes :

```
saveBoard(PrintWriter writer, Board board)
serializeRack(Player p)
```

Format produit :

- Premi re ligne : index du joueur courant (1-based).
- Plateau ligne par ligne : lettre majuscule ou tiret - pour une case vide.
- rack-N: LETTRES — reserve de lettres du joueur N.
- score-N: valeur — score courant du joueur N.
- language en — langue de la partie (non dans [settings]).

Test unitaire :

- **Cas 1 :** Entr e : plateau avec mot COUNT en g5h. Attendu : la ligne g contient C O U N T aux bonnes colonnes.
- **Cas 2 :** Entr e : plateau enti rement vide. Attendu : toutes les cases sont repr sent es par -.
- **Cas 3 :** Entr e : joueur 1 score 15 / joueur 2 score 10. Attendu : pr sence de score-1: 15 et score-2: 10.

Sous besoins 4. Serialiser la section [history]

Description : Ecrire les coups joués dans l'ordre chronologique au format défini par F25.

Fonctions internes :

```
saveHistory(PrintWriter writer, Game game, List<Move> history)
readFullWordFromBoard(Board board, Move move)
convertPointToCoord(Point p)
```

Formats de coup supportés :

- N pass — le joueur passe son tour.
- N exchange ABC — le joueur échange des lettres.
- N g5h WORD ou N e6v WORD — coup normal horizontal ou vertical.

Test unitaire :

- **Cas 1 :** Entree : joueur 1 joue COUNT en g5h, joueur 2 joue PHOTO en e6v. Attendu : présence de 1 g5h COUNT et 2 e6v PHoTO en tête d'historique.
- **Cas 2 :** Entree : aucun coup joué (partie neuve). Attendu : section [history] présente, sans ligne de coup.
- **Cas 3 :** Entree : coup de type pass. Attendu : format N pass écrit correctement.
- **Cas 4 :** Entree : coup de type exchange. Attendu : format N exchange LETTRES écrit correctement.

Sous besoins 5. Charger une partie depuis un fichier

Description : Parser le fichier de sauvegarde et reconstruire un objet Game complet (joueurs, plateau, racks, scores, tour courant et historique).

Fonction associée :

```
Game loadGame(String filePath) throws Exception
```

Actions :

- Prelecture du fichier pour détecter le mode super-scrabble et créer le bon plateau (15×15 ou 21×21).
- Lecture séquentielle et parsing des sections [settings], [game] et [history].
- Reconstruction des joueurs humains et IA avec leurs paramètres.
- Reconstruction de l'historique complet des coups.

Gestion d'erreurs :

- Toute erreur de parsing est remontée sous la forme `Format error at line X: <detail>`.
- La CLI et la GUI affichent ensuite un message utilisateur lisible.
- L'état courant du jeu n'est jamais écrasé en cas d'échec.

Test unitaire :

- **Cas 1 :** Entree : fichier valide et complet. Attendu : Game non nul avec plateau, scores et historique cohérents (`GameLoaderTest.java`).
- **Cas 2 :** Entree : plateau 21×21 (super-scrabble). Attendu : détection correcte et création du bon plateau.
- **Cas 3 :** Entree : fichier contenant `score-1: abc`. Attendu : exception `Format error at line X: ...`.
- **Cas 4 :** Entree : section [game] manquante. Attendu : exception avec message explicite.
- **Cas 5 :** Entree : coup exchange dans l'historique. Attendu : coup restaure correctement.
- **Cas 6 :** Entree : fichier inexistant. Attendu : exception fichier introuvable.

Sous besoins 6. Parser et ignorer les commentaires

Description : Supprimer les commentaires avant le parsing tout en conservant une numérotation de ligne fiable pour les messages d'erreur, conformément à F22.

Fonction interne :

```
processLineComments(String line)
```

Comportement :

- Supprime tout ce qui suit un # sur une ligne (commentaire en ligne).
- Ignore les blocs { ... } potentiellement multi-lignes via un état `isInBlockComment`.
- Continue le parsing ligne par ligne en maintenant le compteur de lignes.

Test unitaire :

- **Cas 1** : Entree : `debug=true` # activer le debug. Attendu : `debug=true` apres suppression du suffixe commente.
- **Cas 2** : Entree : `bloc { commentaire\nsur 2 lignes }` puis `language=fr`. Attendu : bloc supprime, `language=fr` conserve.
- **Cas 3** : Entree : fichier sans commentaire. Attendu : contenu strictement identique en sortie.

Intégration dans l'architecture du projet

Module	Responsabilité	Détail
savefiles	SaveManager.java GameLoader.java	SaveManager : sérialisation de la partie. GameLoader : parsing et reconstruction du Game.
CLI	GameControllerAux.java	<code>save FILE</code> → SaveManager.saveGame <code>load FILE</code> → GameLoader.loadGame Erreurs affichées via <code>cliView.displayError</code>
GUI	ScrabbleGui.java	Menu Save → SaveManager.saveGame Menu Load → GameLoader.loadGame Rebind du contrôleur et rafraîchissement UI après chargement.
Configuration F2	ConfigLoader.java	Lit <code>.scrabblerc</code> dans le HOME. Note : le chemin par défaut de save/load depuis <code>.scrabblerc</code> n'est pas encore pris en charge si FILE est absent.

Scénario utilisateur

1. L'utilisateur joue plusieurs coups en CLI ou via l'interface graphique.
2. En CLI, il tape : `save ma_partie.scrabble`
3. SaveManager écrit `[settings], [game]` puis `[history]` dans le fichier.
4. Le shell affiche un message de succès à l'utilisateur.
5. L'utilisateur relance le programme et tape : `load ma_partie.scrabble`
6. GameLoader reconstruit le Game : joueurs, plateau, tour courant et historique.
7. Si une ligne est invalide, le programme affiche `Format error at line X: ...` et n'écrase pas l'état courant.

A Section en annexe

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.